Gen AI HandsOn- 4

Name: Arnav Sinha
SRN: PES2UG23CS092
Sec: B

1) Exercise Notebook: Multimodal Models

Description of what we did

- Implemented core multimodal AI tasks using text, image, and audio together.
- Performed zero-shot image classification with CLIP by matching images against multiple text labels.
- Generated image captions using BLIP (both general captions and prompt-guided captions).
- Solved visual question answering (VQA) tasks where the model answered questions about image content.
- Used Whisper for speech-to-text transcription and language detection from audio input.
- Built an end-to-end multimodal workflow: audio transcription -> text embedding/search -> image retrieval -> caption generation.

Methods used to solve the problems

- Contrastive image-text alignment (CLIP embeddings + cosine similarity).
- Prompt-based zero-shot classification ("a photo of …" style prompt templates).
- Transformer-based image caption generation (BLIP).
- Transformer-based VQA with image + question conditioning.
- Automatic speech recognition (ASR) with Whisper (log-mel/audio-to-text pipeline).
- Cross-modal retrieval by mapping text and image into a shared embedding space.

Tools used

- Hugging Face Transformers
  (CLIPModel, CLIPProcessor, BlipForConditionalGeneration, BlipForQuestionAnswering, WhisperForConditionalGeneration, WhisperProcessor)
- PyTorch
- Pillow (PIL)
- SoundFile
- Hugging Face Datasets (LibriSpeech)
- Sentence-transformers (embedding support in the pipeline)

2) Exercise Notebook: Agentic Workflows

Description of what we did

- Designed and tested multi-agent systems where each agent had a specialized role.
- Built an AutoGen workflow with researcher, writer, and translator agents communicating sequentially.
- Integrated web search into agents using Tavily results as external context.
- Built an equivalent CrewAI workflow and compared orchestration behavior with AutoGen.
- Implemented a software-team style pipeline (architect -> programmer -> tester -> reviewer).
- Added retry and backoff logic for API rate-limit handling.

Methods used to solve the problems

- Agent role decomposition (breaking one complex task into specialized sub-agents).
- Sequential task passing (output of one agent as input context for the next).
- ReAct-style loop (reason -> act/tool -> observe -> continue).
- Tool-augmented prompting (web search function bound to agents).
- Reliability engineering with exponential backoff on 429/rate-limit responses.
- Comparative framework experimentation (AutoGen vs CrewAI).

Tools used

- AutoGen (pyautogen)
- CrewAI (Agent, Task, Crew, @tool)
- Tavily API client
- Groq-hosted LLMs (Llama variants via OpenAI-compatible endpoints)
- dotenv for API key/environment loading
- Supporting wrappers (LiteLLM/LangChain-compatible APIs as used in notebook)

3) Exercise Notebook: Evaluation, Data, and Ethics

Description of what we did

- Built a RAG pipeline over a curated knowledge base and evaluated answer quality.
- Created test questions (in-scope and out-of-scope/adversarial) to measure robustness.
- Evaluated RAG responses with DeepEval metrics for relevancy and faithfulness.
- Used TruLens for triad-style RAG evaluation (context relevance, groundedness, answer relevance).
- Performed instruction-data quality analysis (length distribution, duplicate pattern checks).
- Demonstrated fairness/bias checks using occupation-gender style prompts.
- Simulated prompt injection attacks and tested safer prompting/sanitization strategies.

Methods used to solve the problems

- Chunking + embedding + FAISS retrieval for context selection.
- LLM-as-judge evaluation (metric-based automated scoring).
- Threshold-based pass/fail analysis for quality assurance.
- Data-centric diagnostics (descriptive statistics + duplicate heuristics).
- Bias probing with controlled prompt templates.
- Red-team style adversarial testing for prompt-injection resilience.

Tools used

- DeepEval (FaithfulnessMetric, AnswerRelevancyMetric, LLMTestCase)
- TruLens (feedback/trace-based RAG evaluation)
- LangChain + FAISS
- Hugging Face/Sentence-transformers embeddings (all-MiniLM-L6-v2)
- Datasets (Alpaca-style instruction dataset usage)
- Pandas, Matplotlib
- Groq-backed chat models for generation and judging

4) Assignment Notebook

Description of what we did

- Developed a full agentic RAG system on a CRISPR-Cas9 knowledge base.
- Verified factual coverage and transformed the knowledge base into retrievable chunks.
- Built a retrieval tool that returns structured context payloads for downstream agents.
- Implemented a 3-stage CrewAI pipeline: answer generation -> evaluation -> revision.
- Tested the system on in-domain and adversarial questions.
- Computed metric-driven performance before and after revision.
- Reported measurable improvement using iterative self-correction.

Methods used to solve the problems

- Domain-specific RAG with chunk overlap and similarity top-k retrieval.
- Structured tool output (JSON context packets) for reliable agent handoff.
- Dual-metric evaluation (faithfulness + answer relevancy).
- Conditional revision loop (revise only when metrics fail threshold).
- Robust API handling with retry/backoff logic.
- Quantitative reporting with pass rates and score comparisons.

Tools used

- CrewAI
- LangChain FAISS + text splitters + Hugging Face embeddings
- DeepEval metrics and custom LLM wrapper usage
- Groq chat models (generation + judge)
- Pandas for result tables/analytics
- Regex/JSON utilities
- dotenv for secure configuration

Conclusion:

Across all four notebooks, the work progressed from foundational multimodal modeling to advanced multi-agent orchestration, then to rigorous evaluation/ethics, and finally to an end-to-end assignment integrating RAG + agentic reasoning + automated quality control. The final assignment demonstrated how combining retrieval, agent collaboration, and metric-based revision can produce more reliable and grounded GenAI outputs.